Doc-Mind: Measuring and Fixing Retrieval Failure on Long PDFs

Ankur Raj - IIT Patna, B.Tech Mathematics and Computing (2028) - October 2026

1. The problem

Doc-Mind is a fully local RAG app (Mistral-7B via Ollama, all-MiniLM-L6-v2 embeddings, FAISS, LangChain LCEL, Gradio). Early testing on long documents exposed a failure that single-document demos hide: once a PDF has many sections that reuse the same vocabulary, a retrieved chunk no longer tells you where it came from. A sentence like *"The pressure relief valve is rated at 3825 psi."* can appear in twelve sections with twelve different values, and dense retrieval cannot tell them apart - so the LLM answers with the wrong section's number. This is the retrieval-side view of the "context rot" that running-summary designs such as *tewtr* (github.com/Rajpragur/tewtr) attack from the generation side.

2. Method: measure first

Before changing anything, I built a harness (`eval_rag.py`) that makes the failure measurable. It generates a synthetic 60-page service manual for a fictional aircraft (12 sections x 5 pages) in which six component types are shared across sections with different spec values, and spec sentences never name their section. 24 questions ask for a spec in a named section; ground truth is the page containing that section's spec sentence. Both retrieval arms use the exact app configuration (all-MiniLM-L6-v2, 1200/300 recursive chunking, cosine similarity, k=5). The only difference is whether each chunk is prepended with a one-line `[document | section | page]` header before embedding. Seed is fixed, so the numbers below reproduce exactly.

3. Results

Retriever	Hit@1	Hit@5	MRR
Baseline (bare chunks)	8.3% (2/24)	62.5% (15/24)	0.278
Context-enriched chunks	33.3% (8/24)	100% (24/24)	0.581

Baseline retrieval loses the correct page entirely (not even in the top 5) on 9 of 24 questions, and 7 of those 9 misses are past page 20: the deeper into the document, the more confounder chunks exist, and the worse bare retrieval gets. The context header removes every top-5 miss. The failure was never *similarity* - the retriever finds near-identical spec sentences easily - it was *disambiguation*, and a single line of context supplies exactly the missing signal.

4. The fix in the app

Context-enriched chunks (default on). At ingest, every chunk is embedded together with its `[file | section | page]` header; section titles are detected per page with a heading heuristic. A checkbox keeps the baseline available for A/B comparison.

Optional LLM running summary. Borrowing tewtr's architecture, the document can be read page by page at ingest while a running summary is carried forward and prepended to each page's chunks, so early definitions survive to page 40. Off by default (slow ingest).

Strict grounding retained. The chain refuses to answer outside the retrieved context and always cites the page.

5. What does not work yet (honest list)

Hit@1 is still only 33%. The right chunk is always in the top 5 now, but ranking it first needs a cross-encoder reranker - not yet implemented.

The benchmark is synthetic. It isolates the near-duplicate-chunk failure mode on purpose; it is not a general RAG benchmark.

The running-summary path is unmeasured. It is implemented, but no dataset isolates its effect yet, so it ships off by default.

Generation quality is not auto-scored. The numbers above are retrieval-only; `eval_rag.py --gen` scaffolds an Ollama-judged answer eval.

One configuration. A single embedding model, chunking config and k were tested; nothing is tuned beyond that.

6. Reproduce

`python eval_rag.py make-data && python eval_rag.py run` - regenerates the manual, runs both arms, and writes eval_data/eval_results.json. Per-question results are in EVALUATION.md.